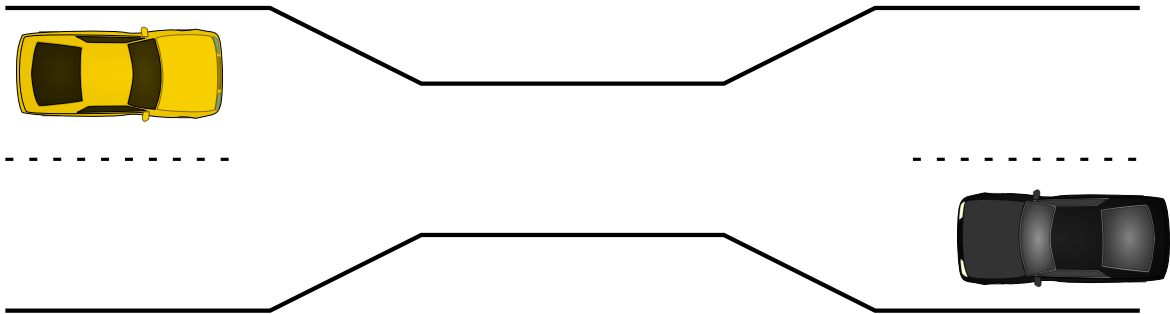
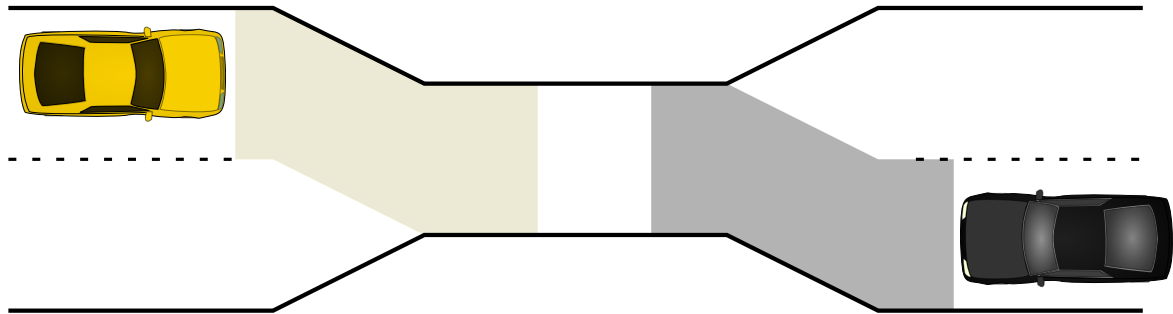


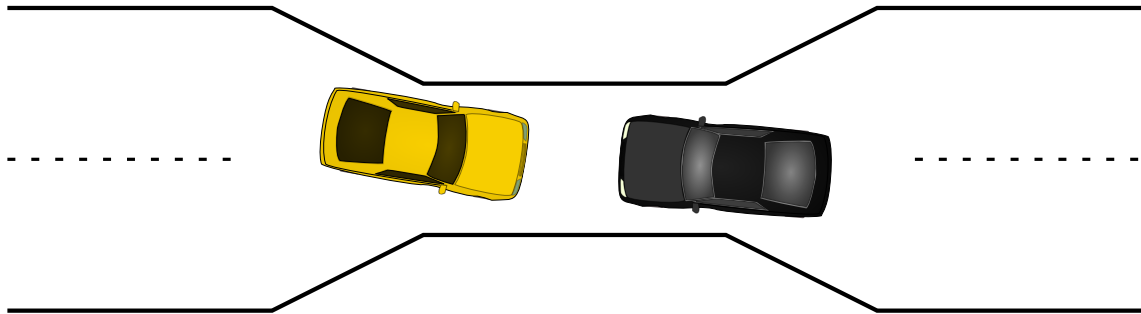
the one-way bridge



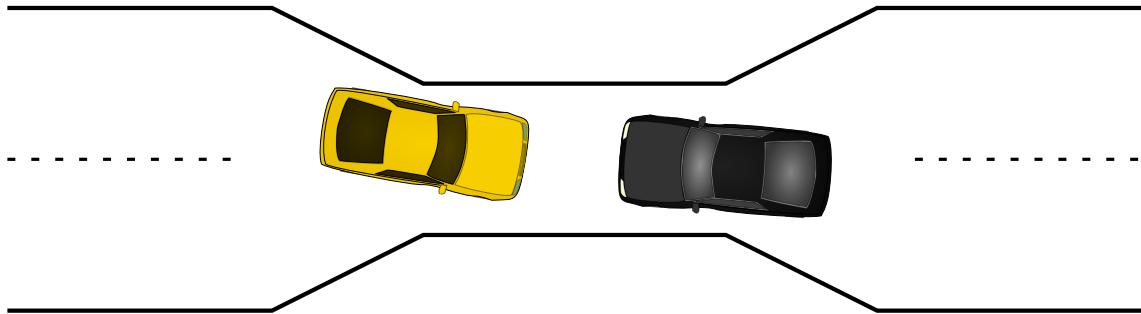
the one-way bridge



the one-way bridge



the one-way bridge



moving two files

```
struct Dir {  
    mutex_t lock; HashMap entries;  
};  
void MoveFile(Dir *from_dir, Dir *to_dir, string filename) {  
    mutex_lock(&from_dir->lock);  
    mutex_lock(&to_dir->lock);  
  
    Map_put(to_dir->entries, filename,  
            Map_get(from_dir->entries, filename));  
    Map_erase(from_dir->entries, filename);  
  
    mutex_unlock(&to_dir->lock);  
    mutex_unlock(&from_dir->lock);  
}
```

Thread 1: MoveFile(A, B, "foo")

Thread 2: MoveFile(B, A, "bar")

moving two files: lucky timeline (1)

Thread 1

MoveFile(A, B, "foo")

lock(&A->lock);

lock(&B->lock);

(do move)

unlock(&B->lock);

unlock(&A->lock);

Thread 2

MoveFile(B, A, "bar")

lock(&B->lock);

lock(&A->lock);

(do move)

unlock(&B->lock);

unlock(&A->lock);

moving two files: lucky timeline (2)

Thread 1

MoveFile(A, B, "foo")

lock(&A->lock);

lock(&B->lock);

(do move)

unlock(&B->lock);

unlock(&A->lock);

Thread 2

MoveFile(B, A, "bar")

lock(&B->lock...

(waiting for B lock)

lock(&B->lock);

lock(&A->lock...

lock(&A->lock);

(do move)

unlock(&A->lock);

moving two files: unlucky timeline

Thread 1

```
MoveFile(A, B, "foo")
```

```
lock(&A->lock);
```

Thread 2

```
MoveFile(B, A, "bar")
```

```
lock(&B->lock);
```

moving two files: unlucky timeline

Thread 1

MoveFile(A, B, "foo")

lock(&A->lock);

lock(&B->lock... *stalled*

(waiting for lock on B)

(waiting for lock on B)

Thread 2

MoveFile(B, A, "bar")

lock(&B->lock);

lock(&A->lock... *stalled*

(waiting for lock on A)

moving two files: unlucky timeline

Thread 1

MoveFile(A, B, "foo")

lock(&A->lock);

lock(&B->lock... *stalled*

(waiting for lock on B)

(waiting for lock on B)

~~(do move)~~ *unreachable*

~~unlock(&B->lock);~~ *unreachable*

~~unlock(&A->lock);~~ *unreachable*

Thread 2

MoveFile(B, A, "bar")

lock(&B->lock);

lock(&A->lock... *stalled*

(waiting for lock on A)

~~(do move)~~ *unreachable*

~~unlock(&A->lock);~~ *unreachable*

~~unlock(&B->lock);~~ *unreachable*

moving two files: unlucky timeline

Thread 1
MoveFile(A, B, "foo")

lock(&A->lock);

lock(&B->lock... *stalled*

(waiting for lock on B)

(waiting for lock on B)

~~(do move)~~ *unreachable*

~~unlock(&B->lock);~~ *unreachable*

~~unlock(&A->lock);~~ *unreachable*

Thread 2
MoveFile(B, A, "bar")

lock(&B->lock);

lock(&A->lock... *stalled*

(waiting for lock on A)

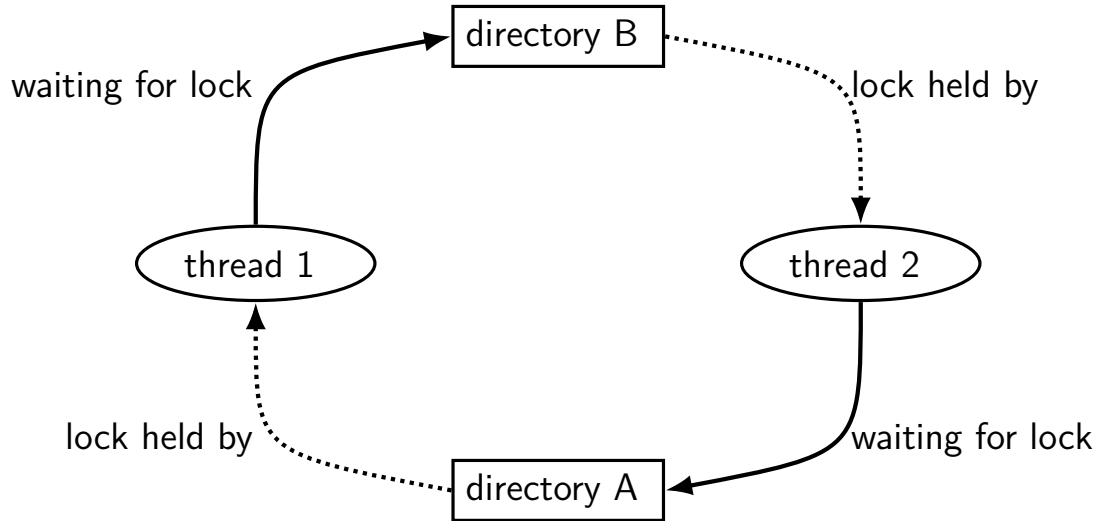
~~(do move)~~ *unreachable*

~~unlock(&A->lock);~~ *unreachable*

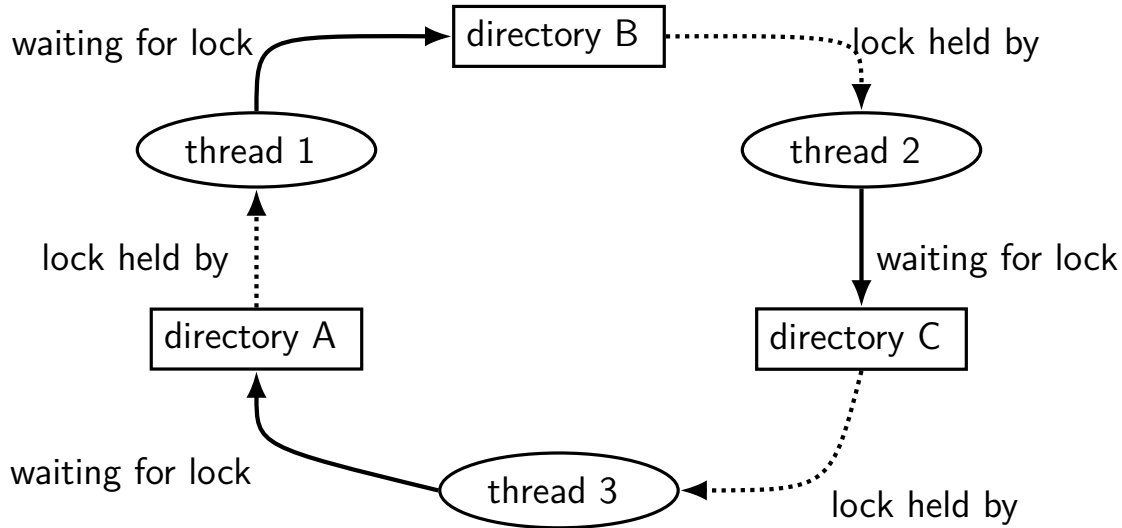
~~unlock(&B->lock);~~ *unreachable*

Thread 1 holds A lock, waiting for Thread 2 to release B lock

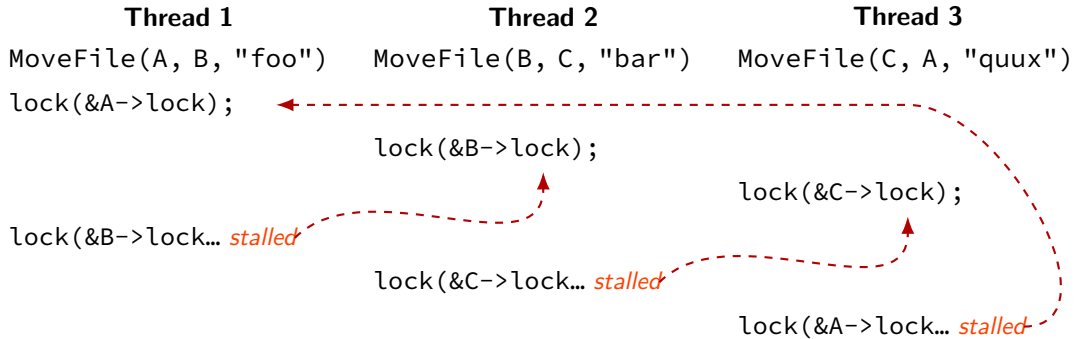
moving two files: dependencies



moving three files: dependencies



moving three files: unlucky timeline



deadlock with free space

Thread 1

AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB)
Free(1 MB)

Thread 2

AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB)
Free(1 MB)

2 MB of space — deadlock possible with unlucky order

deadlock with free space (unlucky case)

Thread 1

AllocateOrWaitFor(1 MB)

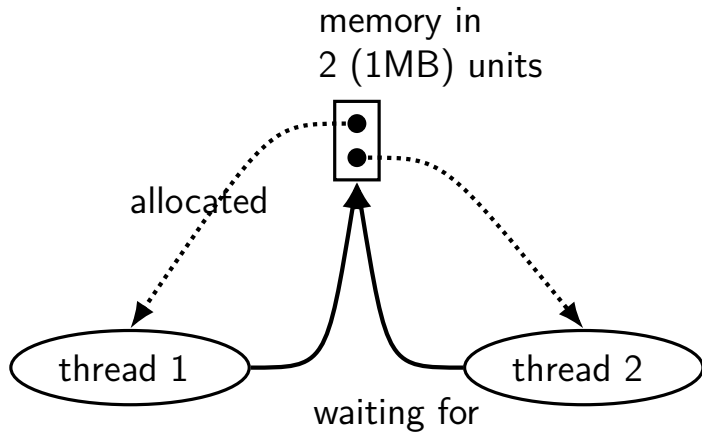
AllocateOrWaitFor(1 MB... *stalled*)

Thread 2

AllocateOrWaitFor(1 MB)

AllocateOrWaitFor(1 MB... *stalled*)

free space: dependency graph



deadlock with free space (lucky case)

Thread 1

```
AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB);
Free(1 MB);
```

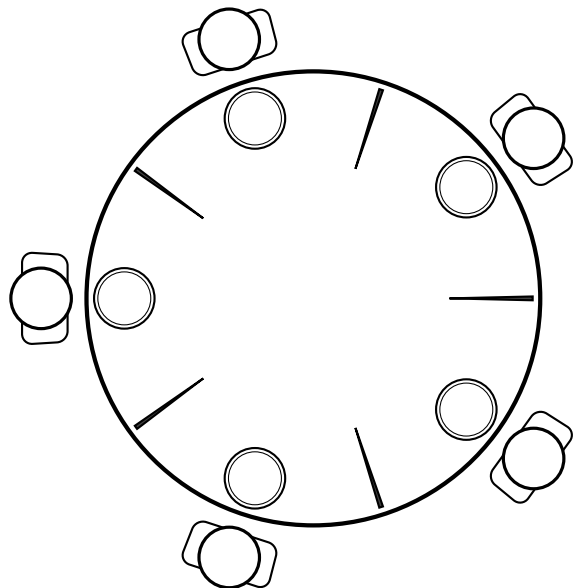
Thread 2

```
AllocateOrWaitFor(1 MB)
AllocateOrWaitFor(1 MB)
(do calculation)
Free(1 MB);
Free(1 MB);
```

lab next week

applying solutions to deadlock to classic *dining philosophers* problem

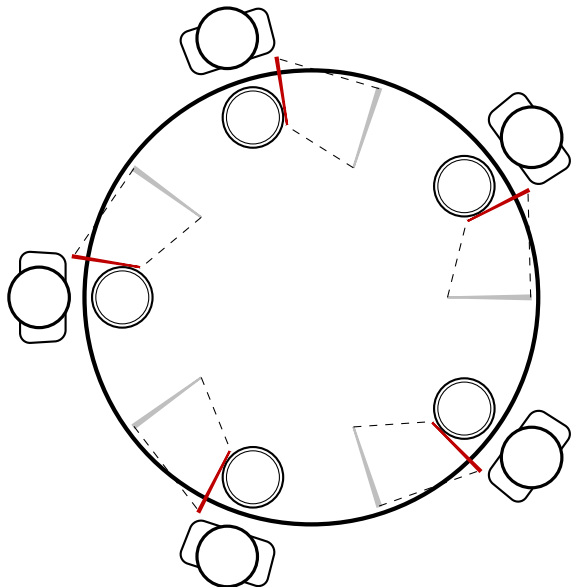
dining philosophers



five philosophers either think or eat
to eat:

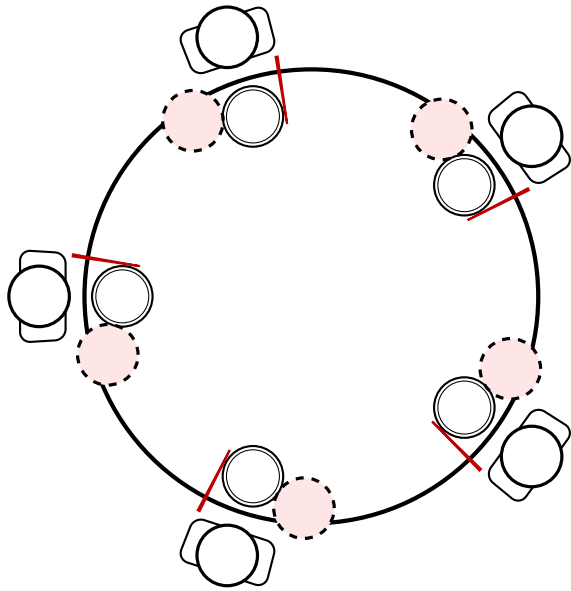
grab chopstick on left, then
grab chopstick on right, then
then eat, then
return chopsticks

dining philosophers



everyone eats at the same time?
grab left chopstick, then...

dining philosophers



everyone eats at the same time?
grab left chopstick, then
try to grab right chopstick, ...
we're at an impasse

deadlock

deadlock — circular waiting for resources

resource = something needed by a thread to do work

- locks

- CPU time

- disk space

- memory

- ...

often non-deterministic in practice

most common example: *when acquiring multiple locks*

deadlock

deadlock — circular waiting for *resources*

resource = something needed by a thread to do work

- locks

- CPU time

- disk space

- memory

- ...

often non-deterministic in practice

most common example: *when acquiring multiple locks*

deadlock requirements

mutual exclusion

one thread at a time can use a resource

hold and wait

thread holding a resources waits to acquire *another* resource

no preemption of resources

resources are only released voluntarily

thread trying to acquire resources can't 'steal'

circular wait

there exists a set $\{T_1, \dots, T_n\}$ of waiting threads such that

T_1 is waiting for a resource held by T_2

T_2 is waiting for a resource held by T_3

...

T_n is waiting for a resource held by T_1

how is deadlock possible?

Given list: A, B, C, D, E

```
RemoveNode(LinkedListNode *node) {  
    pthread_mutex_lock(&node->lock);  
    pthread_mutex_lock(&node->prev->lock);  
    pthread_mutex_lock(&node->next->lock);  
    node->next->prev = node->prev; node->prev->next = node->next;  
    pthread_mutex_unlock(&node->next->lock); pthread_mutex_unlock(&node->prev->lock);  
    pthread_mutex_unlock(&node->lock);  
}
```

Which of these (all run in parallel) can deadlock?

- A. RemoveNode(B) and RemoveNode(C)
- B. RemoveNode(B) and RemoveNode(D)
- C. RemoveNode(B) and RemoveNode(C) and RemoveNode(D)
- D. A and C
- E. B and C
- F. all of the above
- G. none of the above

how is deadlock — solution

Remove B

lock B

lock A (prev)

wait to lock C (next)

Remove C

lock C

wait to lock B (prev)

With B and D — only overlap in in node C — no circular wait possible
(thread can't be waiting while holding something other thread wants)

deadlock prevention techniques

infinite resources

or at least enough that never run out

no mutual exclusion

no shared resources

no mutual exclusion

no waiting

“busy signal” — abort and (maybe) retry
revoke/preempt resources

*no hold and wait/
preemption*

acquire resources in **consistent order**

no circular wait

request **all resources at once**

no hold and wait

deadlock prevention techniques

infinite resources

or at least enough that never run out

no mutual exclusion

no shared resources

no mutual exclusion

no waiting

“busy signal” — abort and (maybe) retry
revoke/preempt resources

*no hold and wait/
preemption*

acquire resources in **consistent order**

no circular wait

request **all resources at once**

no hold and wait

deadlock prevention techniques

infinite resources

or at least enough that never run out

no mutual exclusion

no shared resources

no mutual exclusion

no waiting

“busy signal” — abort and (maybe) retry
revoke/preempt resources

*no hold and wait/
preemption*

acquire resources in **consistent order**

no circular wait

request **all resources at once**

no hold and wait

deadlock prevention techniques

infinite resources

memory allocation: malloc() fails rather than waiting (no deadlock)

locks: pthread_mutex_trylock fails rather than waiting

problem: retry how many times? *no bound on number of tries needed*

...

exclusion

exclusion

no waiting

“busy signal” — abort and (maybe) retry

revoke/preempt resources

*no hold and wait/
preemption*

acquire resources in **consistent order**

no circular wait

request **all resources at once**

no hold and wait

deadlock prevention techniques

infinite resources

or at least enough that never run out

no mutual exclusion

no shared resources

no mutual exclusion

no waiting

“*busy signal*” — *abort and (maybe) retry*

revoke/preempt resources

*no hold and wait/
preemption*

acquire resources in **consistent order**

no circular wait

request **all resources at once**

no hold and wait

deadlock prevention techniques

infinite resources

or at least enough that never run out

no mutual exclusion

no share

requires some way to undo partial changes to avoid errors
common approach for databases

exclusion

no wait

...

“busy signal” — abort and (maybe) retry

*no hold and wait/
preemption*

revoke/preempt resources

acquire resources in **consistent order**

no circular wait

request **all resources at once**

no hold and wait

deadlock prevention techniques

infinite resources

or at least enough that never run out

no mutual exclusion

no shared resources

no mutual exclusion

no waiting

“busy signal” — abort and (maybe) retry
revoke/preempt resources

*no hold and wait/
preemption*

acquire resources in ***consistent order***

no circular wait

request **all resources at once**

no hold and wait

acquiring locks in consistent order (1)

```
MoveFile(Dir* from_dir, Dir* to_dir, string filename) {  
    if (from_dir->path < to_dir->path) {  
        lock(&from_dir->lock);  
        lock(&to_dir->lock);  
    } else {  
        lock(&to_dir->lock);  
        lock(&from_dir->lock);  
    }  
    ...  
}
```

acquiring locks in consistent order (1)

```
MoveFile(Dir* from_dir, Dir* to_dir, string filename) {  
    if (from_dir->path < to_dir->path) {  
        lock(&from_dir->lock);  
        lock(&to_dir->lock);  
    } else {  
        lock(&to_dir->lock);  
        lock(&from_dir->lock);  
    }  
    ...  
}
```

any ordering will do
e.g. compare pointers

acquiring locks in consistent order (2)

often by convention, e.g. Linux kernel comments:

```
/*  
 * ...  
 * Lock order:  
 *     contex.ldt_usr_sem  
 *     mmap_sem  
 *     context.lock  
 */
```

```
/*  
 * ...  
 * Lock order:  
 * 1. slab_mutex (Global Mutex)  
 * 2. node->list_lock  
 * 3. slab_lock(page) (Only on some arches and for debugging)  
 * ...  
 */
```

deadlock prevention techniques

infinite resources

or at least enough that never run out

no *mutual exclusion*

no shared resources

no *mutual exclusion*

no waiting

“busy signal” — abort and (maybe) retry
revoke/preempt resources

no *hold and wait*/
preemption

acquire resources in **consistent order**

no *circular wait*

request ***all resources at once***

no *hold and wait*